Switch Attention: Towards Dynamic and Fine-grained Hybrid Transformers

Yusheng Zhao[♡ ♢], **Hourun Li**[♡ ♢], **Bohan Wu**[♡ ♢], **Yichun Yin**[♣], **Lifeng Shang**[♣], **Jingyang Yuan**[♡ †], **Meng Zhang**[♣ †], **Ming Zhang**[♡ †]

[♡] State Key Laboratory for Multimedia Information Processing, School of Computer Science, PKU-Anker LLM Lab, Peking University, Beijing, China [♣] Huawei Technologies Co., Ltd.

[♢] Contribution during internship at Huawei Technologies Co., Ltd.

{yusheng.zhao,lihourun}@stu.pku.edu.cn, {wxtpku,yuanjy,mzhang_cs}@pku.edu.cn

{zhangmeng92,yinyichun,Shang.Lifeng}@huawei.com

Abstract

The attention mechanism has been the core component in modern transformer architectures. However, the computation of standard full attention scales quadratically with the sequence length, serving as a major bottleneck in long-context language modeling. Sliding window attention restricts the context length for better efficiency at the cost of narrower receptive fields. While existing efforts attempt to take the benefits from both sides by building hybrid models, they often resort to static, heuristically designed alternating patterns that limit efficient allocation of computation in various scenarios. In this paper, we propose Switch Attention (**SwiAttn**), a novel hybrid transformer that enables dynamic and fine-grained routing between full attention and sliding window attention. For each token at each transformer layer, **SwiAttn** dynamically routes the computation to either a full-attention branch for global information aggregation or a sliding-window branch for efficient local pattern matching. An adaptive regularization objective is designed to encourage the model towards efficiency. Moreover, we adopt continual pretraining to optimize the model, transferring the full attention architecture to the hybrid one. Extensive experiments are conducted on twenty-three benchmark datasets across both regular (4K) and long (32K) context lengths, demonstrating the effectiveness of the proposed method. Our kernel implementation is available at https://github.com/YushengZhao/SwitchAttentionKernels.

1 Introduction

The transformer architectures (Vaswani et al., 2017; Dai et al., 2019; Kitaev et al., 2020) have been widely adopted in recent large language models (LLMs) (Anil et al., 2023; OpenAI, 2023; Singh et al., 2025). Despite their success in language modeling, standard transformer architectures suffer from quadratic computational complexity due to the attention mechanism (Vaswani et al., 2017). To address this limitation, efforts have been made in reducing the computation of standard full attention (Shazeer, 2019; Dao et al., 2022; Ainslie et al., 2023; Liu et al., 2024a), sparsifying full attention (Child et al., 2019; Kitaev et al., 2020; Liu et al., 2022a; Zhang et al., 2023; Xiao et al., 2023; Song et al., 2024; Xu et al., 2025; Yuan et al., 2025), developing sub-quadratic attention mechanisms (Katharopoulos et al., 2020; Ma et al., 2021; Han et al., 2023; Peng et al., 2023a; Yang et al., 2024; Gu and Dao, 2024; Peng et al., 2025; Guo et al., 2025a), and designing hybrid transformer models (Beltagy et al., 2020; Zaheer et al., 2020; Jiang et al., 2023; Fu et al., 2024; Du et al., 2025).

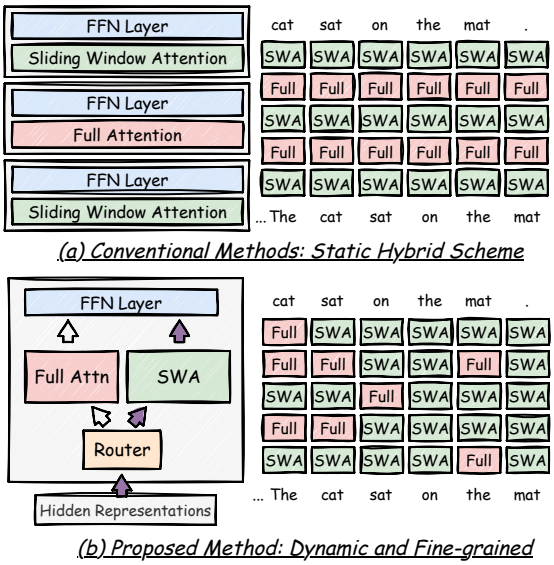


Figure 1: Conventional hybrid transformers (a) often adopt manually designed, static hybrid schemes. In contrast, the proposed **SwiAttn** (b) dynamically selects the type of attention for each input token at each transformer layer, enabling fine-grained allocation of computation.

Among these solutions, hybrid transformer models leverage both standard full attention and its

[†] Corresponding authors.

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 47351–47371
October 24-29, 2026 ©2026 Association for Computational Linguistics

efficient variants, *e.g.*, sliding window attention (SWA, Beltagy et al. 2020), and have been widely adopted in larger industrial models (Gemma, 2024; Yang et al., 2025; OpenAI, 2025; Gao et al., 2026). While they balance efficiency and performance of language modeling, most hybrid transformers adopt a ***static hybrid scheme*** (*e.g.*, alternating between full attention and sliding window attention with a fixed ratio) designed heuristically prior to the pretraining stage (Brown et al., 2020; Jiang et al., 2023; Singh et al., 2025; Du et al., 2025). This limits the model's potential to allocate computation more efficiently. For example, when the context beyond the range of a sliding window is less relevant to the response that the model is about to generate, one would expect a lower ratio of expensive standard full attention, and vice versa.

Towards this end, this paper proposes a novel hybrid attention mechanism named Switch Attention (`SwiAttn`). As is shown in Figure 1, the proposed `SwiAttn` is a ***dynamic and fine-grained*** hybrid solution compared to prior efforts. In particular, `SwiAttn` inserts a router before each attention module and dynamically decides whether the input feature should be processed by standard full attention or sliding window attention for each token at each transformer layer. The full attention branch gathers long-range information accurately from the entire sequence, while the sliding window branch models short-term memory efficiently from a narrower context. The router dynamically decides the specific branch for each token at each layer based on the input feature. In order to encourage the router towards the more efficient branch (*i.e.*, the SWA branch), we adopt a regularization objective that is optimized with the log-likelihood loss. Additionally, to efficiently transfer the knowledge from pretrained full attention models, we utilize continual pretraining (CPT, Sun et al. 2020; Mendieta et al. 2023; Yıldız et al. 2024; Guo et al. 2025b) to convert the standard full attention architecture to the proposed `SwiAttn`. We perform extensive experiments on twenty-three benchmark datasets covering different context lengths against several competing baselines, and the results demonstrate the effectiveness of `SwiAttn`. The contributions of this paper are summarized as follows:

❶ ***New Perspective***: We present a dynamic and fine-grained perspective of hybrid transformer models for effective allocation of computation between full attention and its efficient variants.

❷ ***Novel Methodology***: We propose Switch Attention (`SwiAttn`), consisting of both the full attention branch and the SWA branch, to adaptively allocate computation for each token's representation at each transformer layer. Additionally, we design an adaptive regularization objective and optimize the model from standard transformers through continual pretraining.

❸ ***Empirical Validation***: We conduct extensive experiments on twenty-three benchmark datasets covering commonsense reasoning, in-context retrieval, and long-context understanding. Further experiments reveal the dynamics of the routing, enabling fine-grained allocation of computation.

2 Related Works

2.1 Efficient Attention Mechanism

The quadratic computational complexity has become a major bottleneck of efficient transformer-based LLMs as input sequences grow longer. This leads to a plethora of efforts in designing efficient attention mechanisms (Tay et al., 2022; Sun et al., 2025). One line of research attempts to reduce the computation of the standard full attention itself by sharing (Shazeer, 2019; Ainslie et al., 2023) and compressing (Ge et al., 2023; Liu et al., 2024a,b,c) the key and value vectors, or optimizing hardware compatibility (Dao et al., 2022; Chang et al., 2025; Sanovar et al., 2025). Another line of work designs mechanisms to sparsify the standard full attention through heuristic (Zaheer et al., 2020; Xiao et al., 2023) or data-dependent patterns (Xu et al., 2025; Nawrot et al., 2025; Lai et al., 2025; Yuan et al., 2025).

Several related methods allocate attention or KV-cache computation at different granularities. DuoAttention (Xiao et al., 2025) assigns retrieval or streaming behavior to attention heads, while MoA (Fu et al., 2024) searches heterogeneous attention spans across heads and layers. FastGen (Ge et al., 2023) profiles attention heads and applies head-specific KV-cache compression policies. NSA (Yuan et al., 2025) combines compressed global tokens, selected token blocks, and a local window. CCA-Attention (Chen et al., 2025) constructs core tokens while preserving local context. MInference (Jiang et al., 2024) selects input-dependent sparse attention patterns. In contrast, `SwiAttn` retains exact FullAttn and SWA branches and makes a content-dependent choice for every token at every transformer layer. Recent studies (Wu et al., 2025;

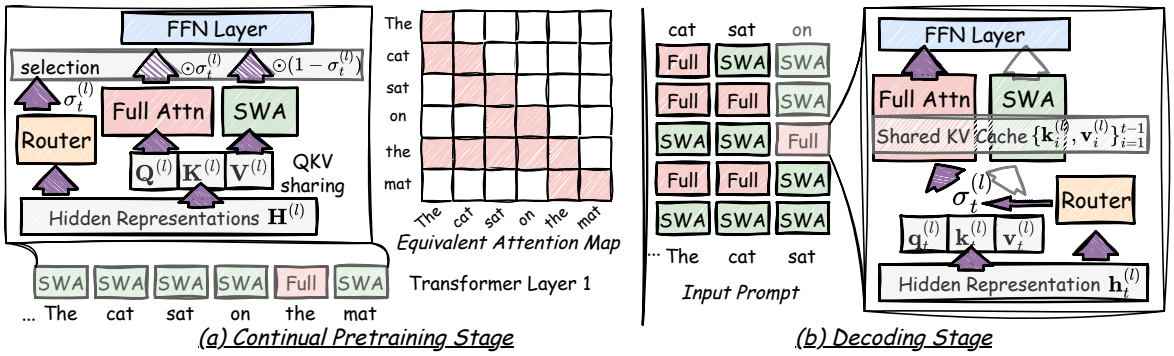


Figure 2: During the continual pretraining stage (a), we compute both the full attention branch and the SWA branch separately. Subsequently, the router is applied to select the attention output from different branches for each token's hidden representation. The selection result is then processed by a feed-forward network (FFN Layer). During the decoding stage (b), the router is used to decide the branch for attention computation. The full attention branch and the SWA branch share a unified KV cache for better efficiency.

Lin et al., 2026) also suggests that the behavior of attention heads may be dynamic.

To circumvent the quadratic complexity, efforts are made in employing sub-quadratic attention (Yang et al., 2024; Han et al., 2024; Dao and Gu, 2024; Peng et al., 2025; Guo et al., 2025a; Kimi et al., 2025). However, their efficiency comes at the expense of degraded long-context performance, as the state matrices limit the memory (Guo et al., 2025a; Nazari and Rusch, 2026; Sun et al., 2026), and in practice, they are often used as an alternative of sliding window attention.

2.2 Hybrid Transformers

Hybrid transformer models have been widely adopted to enhance the efficiency of LLMs (Jiang et al., 2023; Gemma, 2024; Yang et al., 2025). Many hybrid transformers consist of both standard full attention and its efficient variants, such as sliding window (Child et al., 2019), sparse (Zaheer et al., 2020), or linear attention (Yang et al., 2024; Kimi et al., 2025). A common practice among existing designs is to interleave full and efficient attention layers (*e.g.*, Figure 1(a)) according to a static ratio empirically defined before the training process (Wang et al., 2025; OpenAI, 2025; Xiao et al., 2026). Given the heuristic nature of these methods, efforts are made in automated search for optimal hybrid configurations under certain data distributions (Gu et al., 2025; Acun et al., 2025; Xia et al., 2026). Nevertheless, most existing approaches adhere to a static hybrid scheme (using a fixed hybrid pattern for all tokens), either heuristically defined (Kitaev et al., 2020; Kimi et al., 2025; Yang et al., 2025) or searched with data (Gu et al., 2025; Liu and Wang, 2025), while recent works suggest distinct difficulty in generating tokens (Leviathan et al., 2023; Cai et al., 2024; Gloeckle et al., 2024; Liu et al., 2024a; Fu et al., 2025).

3 Methodology

3.1 Preliminaries and Framework Overview

Preliminaries and Notations. We consider the standard language modeling task. During the pretraining stage, the input sentence is first tokenized and then transformed to the hidden representations $\boldsymbol{H}^{(1)} = [\boldsymbol{h}_1^{(1)}, \boldsymbol{h}_2^{(1)}, ..., \boldsymbol{h}_T^{(1)}] \in \mathbb{R}^{d \times T}$, where T is the sequence length and d is the hidden dimension size. At the l-th transformer layer, the input hidden representations $\boldsymbol{H}^{(l)}$ is mapped into queries, keys and values through a linear mapping ϕ, *i.e.*, $[\boldsymbol{Q}^{(l)}, \boldsymbol{K}^{(l)}, \boldsymbol{V}^{(l)}] = \phi(\boldsymbol{H}^{(l)})$. Attention (standard full attention or a more efficient one) is computed using $\boldsymbol{Q}^{(l)}, \boldsymbol{K}^{(l)}, \boldsymbol{V}^{(l)}$ to obtain the attended output $\boldsymbol{O}^{(l)}$. Finally, a feed-forward network is applied to obtain the input hidden representations (*i.e.*, $\boldsymbol{H}^{(l+1)}$) of the next transformer layer. The final hidden representation $\boldsymbol{H}^{(L+1)}$ (where L is the number of transformer layers) is used to predict the next token. During the decoding stage, the model generates one token each time with the current query $\boldsymbol{q}_t^{(l)}$, key $\boldsymbol{k}_t^{(l)}$ and value $\boldsymbol{v}_t^{(l)}$, and uses KV cache $\{\boldsymbol{k}_i^{(l)}, \boldsymbol{v}_i^{(l)}\}_{i=1}^{t-1}$ to store previously computed key and value vectors. [1]

Framework Overview. The proposed `SwiAttn` is a dynamic and fine-grained hybrid of standard full attention and SWA, as illustrated in Figure

[1] For simplicity, we omit some details here and in the following, such as layer normalization, residual connection, position embeddings, attention heads, *etc.*

2. At each transformer layer during the continual pretraining stage, both full attention and SWA are computed using the shared queries, keys and values. A router is applied to the input hidden representations to decide which branch to use for each token. The attended outputs from the two branches are then gathered and processed by a feed-forward layer. An adaptive regularization objective is applied to the output of each router to encourage the model towards the SWA branch. During the decoding stage, the router is first applied to the hidden representation to decide the branch for computing, and the model only computes the selected branch using a shared KV cache.

3.2 Dual-branch Routing

The proposed `SwiAttn` consists of two branches: the full attention branch and the SWA branch. Standard full attention excels in capturing long-range dependencies, while SWA efficiently computes attended output using local contexts. Not all tokens require global information at all transformer layers: there is a natural sparsity in both layer (Yang et al., 2025; Kimi et al., 2025; Gao et al., 2026) and token (Leviathan et al., 2023; Cai et al., 2024; Fu et al., 2024, 2025) dimensions. Therefore, we use a router function to decide the specific branch to use.

Concretely, given the input hidden representation $\boldsymbol{H}^{(l)} \in \mathbb{R}^{d\times T}$ at the l-th layer, we first compute the soft gate using the router function $\mathcal{R}$ as:

$$\tilde{\sigma}^{(l)} = \text{sigmoid}\left(\mathcal{R}(\boldsymbol{H}^{(l)})\right) \in [0,1]^T, \quad (1)$$

where $\text{sigmoid}(\cdot)$ is the sigmoid function. The (hard) gate indicating the path of computation for each token at the l-th layer, $\sigma^{(l)} \in \{0,1\}^T$, is then obtained by applying a threshold $\tau = 0.5$, *i.e.*,

$$\sigma^{(l)} = \mathbb{1}\left(\tilde{\sigma}^{(l)} > \tau\right) + \tilde{\sigma}^{(l)} - \text{sg}\left(\tilde{\sigma}^{(l)}\right), \quad (2)$$

where $\mathbb{1}(\cdot)$ is an element-wise indicator function and $\text{sg}(\cdot)$ is the stop gradient function. Note that we adopt straight-through estimation (STE, Fedus et al. 2022; Liu et al. 2022b; Huh et al. 2023) to enable differentiation of the router's parameters during training. The gate is used to control whether a corresponding token should be computed with full (when the value is 1) or sliding window attention (when the value is 0). Subsequently, we compute both the full attention branch and the SWA branch using a shared set of queries, keys and values, *i.e.*,

$$[\boldsymbol{Q}^{(l)}, \boldsymbol{K}^{(l)}, \boldsymbol{V}^{(l)}] = \phi(\boldsymbol{H}^{(l)}), \quad (3)$$

$$\boldsymbol{O}^{(l)}_{\text{FULL}} = \mathcal{A}^{(l)}_{\text{FULL}}(\boldsymbol{Q}^{(l)}, \boldsymbol{K}^{(l)}, \boldsymbol{V}^{(l)}), \quad (4)$$

$$\boldsymbol{O}^{(l)}_{\text{SWA}} = \mathcal{A}^{(l)}_{\text{SWA}}(\boldsymbol{Q}^{(l)}, \boldsymbol{K}^{(l)}, \boldsymbol{V}^{(l)}), \quad (5)$$

where $\mathcal{A}^{(l)}_{\text{FULL}}$ and $\mathcal{A}^{(l)}_{\text{SWA}}$ denote full attention and SWA, respectively. The gate is then used to select the output of attention $\boldsymbol{O}^{(l)}$, *i.e.*,

$$\boldsymbol{O}^{(l)} = \sigma^{(l)} \odot \boldsymbol{O}^{(l)}_{\text{FULL}} + (1-\sigma^{(l)}) \odot \boldsymbol{O}^{(l)}_{\text{SWA}}, \quad (6)$$

where $\odot$ denotes element-wise product (in the sequence length dimension). Finally, the feed-forward network (FFN) is applied to obtain the hidden representation at the $(l+1)$-th layer.

During inference, the attention output is computed similarly. At the prefill stage, the attention output can be computed via Eq.1-6, and especially, the computation of the two branches could be further merged with a hardware-efficient implementation. At the decode stage, we compute the (hard) gate for the t-th token at the l-th layer as a scaler $\sigma^{(l)}_t \in \{0,1\}$, and then compute the full attention or SWA accordingly. Notably, in the decoding stage, both branches share a single set of KV-cache to increase inference efficiency. We summarize the inference procedure of `SwiAttn` in Appendix A.

3.3 Adaptive Regularization

In the dual-branch routing framework, the full attention branch provides richer information than the SWA branch. Therefore, to encourage the model towards the SWA branch for efficiency, we also design an adaptive regularization objective that is optimized during continual pretraining. Conceivably, when the ground-truth token exhibits lower log-likelihood under the predicted distribution, the model may struggle to predict this token, and therefore, we expect higher ratios for the more powerful full attention branch. Additionally, when the two branches disagree, we also tend toward the more reliable full attention branch. These require the regularization objective to be more adaptive.

Specifically, given the soft gate of the t-th token at the l-th transformer layer $\tilde{\sigma}^{(l)}_t$, the regularization objective is designed as follows:

$$\mathcal{L}^{(l)}_t = \gamma^{(l)}_t \, \text{sp}\left(\mathcal{R}(\boldsymbol{h}^{(l)}_t)\right), \quad (7)$$

where $\gamma^{(l)}_t$ is the adaptive weight of regularization discussed in the following, and $\text{sp}(x) = \ln(1+e^x)$

is the softplus function, a smooth version of the ReLU function (Nair and Hinton, 2010), providing a small penalty to values less than 0 and increasing penalty to values greater than 0. The adaptive weight $\gamma_t^{(l)}$ of the t-th token at the l-th layer is determined by several factors: (1) the base weight γ_{base}, (2) the negative log-likelihood (NLL) of the ground-truth token under the currently predicted distribution of language modeling, *i.e.*, $\text{NLL}(t)$, and (3) the disagreement of the two branches as measured by the mean squared error (MSE), *i.e.*, $\|\boldsymbol{o}_{\text{FULL}}^{(t,l)} - \boldsymbol{o}_{\text{SWA}}^{(t,l)}\|_2^2$, where $\boldsymbol{o}_{\text{FULL}}^{(t,l)}$ and $\boldsymbol{o}_{\text{SWA}}^{(t,l)}$ denotes the output of the full attention branch and the SWA branch of the t-th token at the l-th layer. Considering these factors, the adaptive weight $\gamma_t^{(l)}$ can be written as follows:

$$\gamma_t^{(l)} = \frac{\gamma_{\text{base}}}{\varepsilon + \text{NLL}(t) + \alpha \cdot \|\boldsymbol{o}_{\text{FULL}}^{(t,l)} - \boldsymbol{o}_{\text{SWA}}^{(t,l)}\|_2^2}, \tag{8}$$

where ε and α are hyper-parameters.

The NLL and branch MSE are used only to construct the adaptive regularization weight during teacher-forced CPT. At inference, the routing decision depends only on $\mathcal{R}(\boldsymbol{H})$, and the model computes only the selected attention branch.

Remark. Eq.8 adaptively reweights the regularization objective of the router. The hyperparameter ε avoids division by zero and provides an upper bound of the weight, *i.e.*, $\gamma_t^{(l)} \leq \gamma_{\text{base}} \cdot \varepsilon^{-1}$. The NLL and MSE terms reduce the weight of regularization when the token prediction loss increases (high NLL) and/or the two branches diverge (high MSE). Under such conditions, the reduced regularization weights encourage the router to use fewer SWA branches and more full attention branches, since the SWA branch may be less reliable. Although the full-attention ratio emerges from training, it can be steered toward a target compute budget by adjusting γ_{base} according to the observed routing ratio: increasing γ_{base} encourages more SWA decisions, whereas decreasing it relaxes the efficiency-oriented penalty. The experiments in this work use a fixed γ_{base}. Further motivation for this design is provided in Appendix C.

3.4 Continual Pretraining

Different from many hybrid transformers that require training from scratch, the proposed `SwiAttn` adopts a continual pretraining approach, utilizing the learned knowledge from an existing off-the-shelf full attention transformer.

Algorithm 1 The continual pretraining of `SwiAttn`

Requires: The pretrained standard full attention transformer model $\mathcal{M}^{pre}$, hyperparameter ε and α, number of continual pretraining iterations N.

Ensures: The continual pretrained `SwiAttn` $\mathcal{M}$.

```
Initialize the weights of A^(l)_FULL, A^(l)_SWA in M using the weights Ã^(l)_FULL from M^pre;
for Iterations i = 1, 2, ..., N do
    Tokenize the training batch of the i-th iteration and transform them into initial hidden representations H^(1);
    for Layers l = 1, 2, ..., L do
        Compute the soft gate σ̃^(l) using Eq.1;
        Obtain the hard gate σ^(l) using threshold τ according to Eq.2;
        Compute the routed output O^(l) using Eq.3-6 and apply the FFN layer;
        Compute the router logits R(h_t^(l)) and MSE ||o^(t,l)_FULL − o^(t,l)_SWA||_2^2 for each token;
    end for
    Compute the NLL for each token as NLL(t);
    Compute the adaptive weight γ_t^(l) using Eq.8 to obtain adaptive regularization L_t^(l);
    Compute the language modeling loss L_LM and the final loss L using Eq.9;
end for
Update the parameters of M using ∇L;
```

In particular, given a standard transformer and its full attention module at the l-th layer $\tilde{\mathcal{A}}_{\text{FULL}}^{(l)}$, we initialize parameters of both the full attention branch $\mathcal{A}_{\text{FULL}}^{(l)}$ and the SWA branch $\mathcal{A}_{\text{SWA}}^{(l)}$ of `SwiAttn` using the parameters of $\tilde{\mathcal{A}}_{\text{FULL}}^{(l)}$. Other parameters, like the FFN layer, are also inherited from the standard pretrained transformer. During the continual pretraining stage, we train the entire hybrid transformer using both the language modeling objective $\mathcal{L}_{\text{LM}}$ and the aforementioned adaptive regularization objective, *i.e.*,

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \frac{1}{LT} \sum_{l=1}^{L} \sum_{t=1}^{T} \mathcal{L}_t^{(l)}. \tag{9}$$

By continual pretraining from an existing standard transformer, we save computation resources and stabilize the optimization process. The continual pretraining process of the proposed `SwiAttn` is presented in Algorithm 1.

3.5 Hardware Compatibility

During CPT, `SwiAttn` computes both full attention and SWA. The SWA branch can be implemented efficiently using FlashAttention (Dao et al., 2022). The router maps each hidden representation to a scalar and can be fused into the QKV projection as one additional output, making its overhead negligible relative to attention.

During inference, efficient attention does not automatically translate into efficient hardware execution. For example, in Q-outer sparse attention under GQA, Lai et al. (2026) derive its BF16 arithmetic intensity. Using G for the GQA group size and S for the number of KV tokens processed by a query, its arithmetic intensity can be written as

$$\mathrm{AI}_{\text{Q-outer}} = \frac{GS}{G+S} \approx G, \qquad S \gg G. \quad (10)$$

Thus, the arithmetic intensity is largely determined by the GQA group size, and many Q-outer sparse-attention kernels require a sufficiently large G for efficient execution (Yuan et al., 2025; Liu et al., 2025; Zeng et al., 2026).

`SwiAttn` instead groups routed query positions into branch-specific, position-sorted index lists and gathers them in tiles of B_q positions for each query head. For the G query heads sharing one KV head, the kernel processes each query head in a separate thread block, so both computation and the issued KV loads scale with G under the same IO accounting. For a query tile traversing S KV tokens, its dominant BF16 arithmetic intensity, ignoring lower-order traffic, is

$$\begin{aligned}\mathrm{AI}_{\texttt{SwiAttn}} &= \frac{4GB_qSd_h}{4GB_qd_h + 4GSd_h} = \frac{B_qS}{B_q+S} \\ &\approx B_q, \qquad S \gg B_q.\end{aligned} \quad (11)$$

Thus, KV reuse is determined by the kernel-controlled query-tile size B_q, rather than the GQA group size G. `SwiAttn` consequently avoids the large-group requirement common to many sparse-attention methods (Yuan et al., 2025; Liu et al., 2025; Zeng et al., 2026). This makes `SwiAttn` an efficient and hardware-friendly attention mechanism under smaller group sizes. An exemplar kernel implementation is provided in Appendix B.

During decoding, `SwiAttn` remains compatible with existing optimization infrastructure, including PagedAttention (Kwon et al., 2023) and Split-KV strategies such as FlashInfer (Ye et al., 2025).

4 Experiments

4.1 Experimental Setups

Datasets. Three types of datasets are used. ▶ *Commonsense Reasoning*, including Lambada (LMB., Paperno et al. 2016), Wikitext (Wiki., Merity et al. 2016), BoolQ (Clark et al., 2019), PIQA (Bisk et al., 2020), SIQA (Sap et al., 2019), HellaSwag (Hella., Zellers et al. 2019), WinoGrande (Wino., Sakaguchi et al. 2021), ARC-easy and ARC-challenge (ARC-e and ARC-c, Clark et al. 2018), and OpenBookQA (OBQA, Mihaylov et al. 2018); ▶ *In-Context Retrieval* from Arora et al. 2024, including SWDE (Lockard et al., 2019), FDA (Arora et al., 2023), TriviaQA (TQA, Joshi et al. 2017), NQ (Kwiatkowski et al., 2019), DROP (Dua et al., 2019), and SQuAD (SQD, Rajpurkar et al. 2018); ▶ *Long-Context Understanding* from LongBench-E (Bai et al., 2024), including MultiFieldQA (MFQA, Bai et al. 2024), TRec (Li and Roth, 2002), MultiNews (MNs, Fabbri et al. 2019), GovReport (GvR, Huang et al. 2021), Qasper (Qas, Dasigi et al. 2021), TriviaQA (TQA, Joshi et al. 2017), and SAMSum (SSM, Gliwa et al. 2019). More details about the datasets are provided in Appendix D.

Baselines. We compare the proposed `SwiAttn` against several baselines: sliding window attention in zero-shot and continual-pretraining forms, denoted SWA-ZS and SWA-CPT; static hybrids of full attention and SWA commonly used in larger industrial LLMs (Brown et al., 2020; OpenAI, 2025; Xiao et al., 2026), denoted SH (3:1) and SH (6:1) according to their SWA-to-FullAttn layer ratios; DuoAttention (Xiao et al., 2025); and full attention, denoted FullAttn. More details about the baseline methods are provided in Appendix E.

Implementation Details. For the transformer architecture, we use a 1.5B language model with 22 transformer layers (*i.e.*, $L = 22$). Each layer contains a dual-branch routing module and an FFN layer. The router function $\mathcal{R}(\cdot)$ is implemented with a linear mapping, and the threshold τ is set to 0.5. For the SWA branch, the window size is set to 1024. In adaptive regularization, we set the base weight γ_{base} to 1×10^{-3}, the ε to 0.1 and the weight α to 100. For model optimization, we first pretrain the 1.5B full attention model on 4K-length sequences using 240B tokens and then continually pretrain the `SwiAttn` model for 20B tokens. To evaluate tasks of longer sequences, we optimize

Table 1: Performance on common sense reasoning datasets. Best results are in **bold** and second-best underline.

Datasets	LMB.	Wiki.	AVG	BoolQ	PIQA	SIQA	Hella.	Wino.	ARC-e	ARC-c	OBQA	AVG
Metrics	ppl↓	ppl↓	ppl↓	acc↑	acc↑	acc↑	acc_n↑	acc↑	acc↑	acc_n↑	acc_n↑	acc↑
SWA-ZS	7.16	16.6	11.9	62.2	74.0	41.4	60.1	57.1	62.9	34.3	35.4	50.5
SWA-CPT	**7.02**	15.6	11.3	62.1	**74.5**	42.0	60.1	**57.9**	62.7	**35.5**	35.2	52.6
SH (3:1)	7.32	15.2	11.3	**62.3**	73.7	42.0	**60.2**	**57.9**	62.3	33.6	36.7	53.6
FullAttn	7.16	**15.0**	**11.1**	62.2	74.0	41.4	60.1	57.1	62.9	34.3	35.4	53.4
`SwiAttn`	7.28	**15.0**	**11.1**	62.0	74.0	**42.3**	**60.2**	57.8	**63.0**	34.0	**36.8**	**53.7**

Table 2: Performance on in-context retrieval datasets. Best results are in **bold** and second-best underline.

Datasets	SWDE	FDA	TQA	NQ	DROP	SQD	AVG
SWA-ZS	52.0	22.5	64.6	30.9	31.1	49.2	41.7
SWA-CPT	49.1	24.0	**65.7**	28.4	32.2	49.7	41.5
SH (3:1)	56.5	67.9	65.1	32.7	31.3	**50.1**	50.6
FullAttn	**60.8**	**76.4**	64.6	35.1	31.1	49.2	**52.9**
`SwiAttn`	60.2	75.1	65.2	**35.2**	**32.4**	49.4	**52.9**

Table 3: Performance on long-context understanding. Best results are in **bold** and second-best underline.

Datasets	GvR	MNs	MFQA	Qas	TQA	SSM	TRec	AVG
SWA-ZS	19.2	18.2	18.5	10.5	51.1	27.4	18.7	23.4
SWA-CPT	19.2	18.7	24.2	13.1	57.8	30.7	53.0	31.0
DuoAttention	21.6	19.9	30.9	14.5	62.4	29.4	58.7	33.9
SH (6:1)	21.0	19.0	26.2	**20.1**	60.0	31.2	55.7	33.3
SH (3:1)	22.6	20.0	29.6	14.5	68.1	32.3	59.3	35.2
FullAttn	24.9	20.9	33.9	18.6	69.3	30.2	**63.7**	**37.4**
`SwiAttn`	**25.0**	**21.3**	**35.3**	18.0	**69.9**	**32.4**	60.0	**37.4**

the 32K full attention model by pretraining on 4K context lengths for 192B tokens and then extending the context length using YaRN (Peng et al., 2023b) to 32K with 24B tokens. The corresponding `SwiAttn` is obtained by continual pretraining on the full attention model for another 8B tokens. More details can be found in Appendix F.

4.2 Main Results

We compare the performance of `SwiAttn` against baselines on 23 benchmark datasets, and the results are shown in Table 1, Table 2 and Table 3.

Commonsense Reasoning (Table 1). As the contexts of most instances are within the sliding window size (*i.e.*, 1024), the performance of `SwiAttn` is similar to the baselines, including both full attention and SWA models, demonstrating that ***Obs.❶ `SwiAttn` maintains the commonsense knowledge of full attention after CPT.***

In-Context Retrieval (Table 2). For these datasets, the contexts are within the 4096 token limit of 4K-length models, and some instances require attention spans beyond the sliding window. ***Obs.❷ `SwiAttn` matches FullAttn while outperforming SH (3:1) and the SWA baselines.*** This corresponds to a relative improvement of 4.5% over SH (3:1).

Long-Context Understanding (Table 3). For these datasets, the contexts of most instances grow beyond the 1024 window size and can be as long as 32K, which requires strong long-context capability. ***Obs.❸ `SwiAttn` matches FullAttn on average while outperforming DuoAttention, both StaticHybrid configurations, and the SWA baselines.*** Compared with SH (3:1), `SwiAttn` achieves a relative improvement of 6.3%, despite using full attention less frequently (as shown in Section 4.4).

Overall, `SwiAttn` achieves performance parity with FullAttn while improving over the other baselines on tasks that require longer contexts. Results on 3B models are in Appendix I.

4.3 Ablation Studies

To investigate the effect of different mechanisms in the proposed method, we design several variants of the original `SwiAttn`. Specifically, V1 disables adaptive regularization and uses γ_{base} instead. V2 removes the NLL term in Eq.8. V3, V4, and V5 changes α from 100 to 10, 1, and 0, respectively. V6 and V7 perturb ε from 0.1 to 0.2 and 0.01. We present results on three summarization datasets from LongBench-E in Table 4.

Effect of Adaptive Regularization. From V1, V2, and V5, we can see that ***Obs.❹ adaptive regularization, along with several factors in adaptive weights, is effective***. The removal of adaptive regularization reduces average performance by 1.3. Besides, when the NLL (V2) or MSE (V5) terms are removed, the performance also declines.

Effects of Hyperparameters. V3 and V4 show the effect of the hyperparameter of α. When α is changed, the accuracy drops, as α balances the relative importance of the MSE term in adaptive regularization. Variants V6 and V7 show the effect of ε. The model is less sensitive to this hyperparameter, and when ε is perturbed, the accuracy decreases slightly. ε controls the upper bound of the adaptive weights, which affects the optimization process.

Table 4: Ablation studies of **SwiAttn**.

Variants	GvR	MNs	SSM	AVG
V1: *w/o* Adaptive Reg.	24.1	20.7	30.0	24.9
V2: *w/o* the NLL term	24.3	20.9	30.7	25.3
V3: $\alpha = 10$	24.1	20.9	30.5	25.2
V4: $\alpha = 1$	23.8	20.9	29.9	24.9
V5: $\alpha = 0$	23.9	20.6	29.5	24.7
V6: $\varepsilon = 0.2$	24.7	20.9	31.7	25.8
V7: $\varepsilon = 0.01$	24.4	20.8	30.1	25.1
SwiAttn	25.0	21.3	32.4	26.2

4.4 Further Analysis

Needle-in-a-Haystack. We also provide experimental results on the commonly-used Needle-In-A-Haystack (NIAH) task using RULER (Hsieh et al., 2024). Specifically, we test 35 different context positions (*i.e.*, depth percent) and 35 context lengths (*i.e.*, token limit). The retrieval tests are repeated three times for each case, and the results are visualized in Figure 3. As can be seen from the results, ***Obs.❺ SwiAttn achieves perfect retrieval accuracy across various scenarios***, showing that **SwiAttn** maintains the strong retrieval capability of full attention with its dynamic routing mechanism models under long context lengths.

Inference Efficiency. We compare the inference efficiency of **SwiAttn** and the FullAttn baseline. For the prefill stage, we plot the amount of computation required (in terms of GFLOPs) at every token position, following Liu et al. 2025, and the results are shown in Figure 4a. For the decode stage, we measure efficiency with the amount of memory access of the KV cache (in terms of the number of tokens at each layer on average), as it is the primary bottleneck of the decoding speed of attention. As the efficiency of **SwiAttn** is related to the data distribution, we provide results on both standard (under the same distribution of training corpus) and simple (under the distribution of needle-in-a-haystack task where the needle is within the sliding window) setups in Figure 4b. At a 32K context length, **SwiAttn** reduces KV-cache access by approximately $4.4\times$ under the standard data distribution and by up to $12\times$ under the simple data distribution.

Ratios of Full Attention across Layers. We then investigate how different transformer layers behave in dual-branch routing. Specifically, we collect the routing decisions of each transformer layer and calculate the ratio of the model choosing full attention under the 32K continual pretraining data

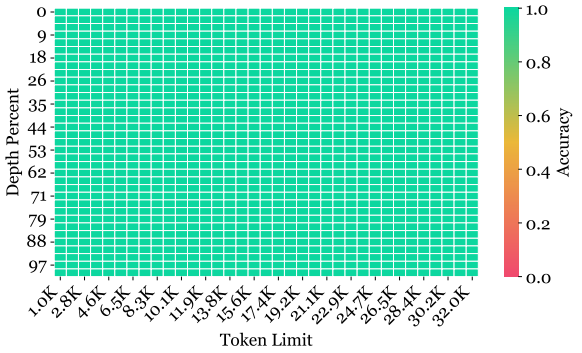
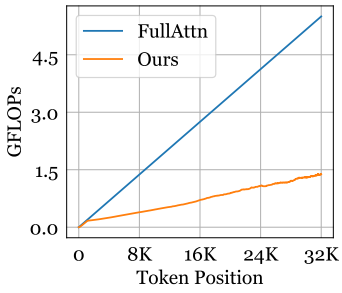


Figure 3: Needle-in-a-haystack retrieval accuracy in different context positions (depth percentage) across 32K context lengths. The proposed **SwiAttn** achieves perfect retrieval accuracy with its dynamic and fine-grained hybrid attention mechanism.



(a) Prefill

Pos.	Mem. Access Ours	Full
Standard		
8K	2.3K	8K
16K	4.2K	16K
24K	6.2K	24K
32K	7.3K	32K
Simple		
16K	1.7K	16K
32K	2.6K	32K

(b) Decode

Figure 4: Efficiency comparison under different token positions of **SwiAttn** (ours) and FullAttn baseline. We report GFLOPs for the prefill stage and memory access in terms of averaged tokens in the decode stage.

distribution and count the decisions of 400M tokens. The results are presented in Figure 5. As can be seen from the results, ***Obs.❼ different transformer layers exhibit distinct patterns of routing***: some layers (*e.g.*, Layer 1, 2, 3, 22) tend to use more SWA branches while others (*e.g.*, Layer 8, 11, 14) use full attention branches more frequently. This suggests different layers specialize in different functionalities. Additionally, we also compute the averaged full attention ratio of all layers, and the result is around 0.13, which is lower than the 0.25 ratio of conventional static hybrid models with one full attention layer after three SWA layers.

Dynamics in Dual-Branch Routing. We also investigate the dynamics of dual-branch routing. When the next token can be easily predicted based on information within the sliding window, we expect the model to use the SWA branch more frequently, and when the next token is hard to forecast with local information, the full attention branch should be more frequently used as a default choice.

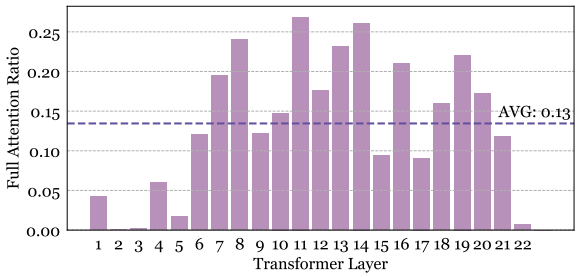


Figure 5: The ratio of using full attention branches across different transformer layers. The averaged full attention ratio is around 0.13.

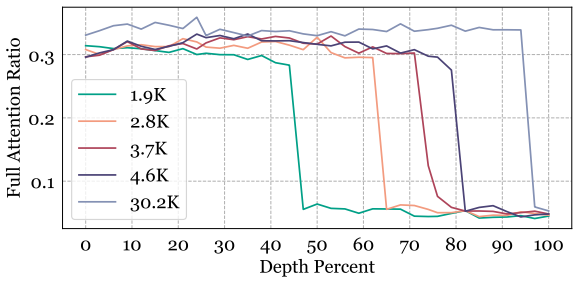


Figure 6: The ratio of full attention branches *when generating the needle* in the needle-in-a-haystack retrieval task. When the needles fall within the sliding window, the full attention ratio is low, while a higher ratio is observed when the needle is outside the sliding window.

To verify whether `SwiAttn` behaves in a computation efficient way, we resort to the NIAH retrieval task, where needles to be retrieved are inserted in different locations of the context (*i.e.*, different depth percent, higher percents indicate closer distance to the end of the prompt), and measure the full attention ratio of `SwiAttn` when generating the answer (*i.e.*, the inserted needle). The results are presented in Figure 6, where we illustrate the ratio under different context lengths (*i.e.*, 1.9K, 2.8K, 3.7K, 4.6K, 30.2K). The results show that when the needle is inserted in lower depth percentages (outside the sliding window), the model chooses around 30% of full attention, while when the model is closer to the end of the prompt (within the sliding window), a lower ratio of full attention around 5% is observed. This demonstrates that ***Obs.❸ `SwiAttn` can dynamically and efficiently decide the specific branch to use based on the context***.

5 Conclusion

This paper presents Switch Attention (`SwiAttn`), a novel hybrid transformer that can dynamically route the hidden representation to either full attention or sliding window attention at each transformer layer, enabling efficient and fine-grained allocation of computation based on the contexts. `SwiAttn` can be optimized via continual pretraining from off-the-shelf standard full attention transformers. An adaptive regularization objective is designed to enhance efficiency while maintaining accuracy. Extensive experiments demonstrate the effectiveness of the proposed `SwiAttn`, and further empirical results reveal its dynamics and efficiency.

Limitations

Our experiments cover models up to 3B parameters, and the 3B model uses the same hyperparameter configuration as the 1.5B model without a scale-specific search. Effectiveness at larger industrial scales therefore remains to be verified. In addition, the efficiency of `SwiAttn` depends on the routing distribution induced by the input. In the worst case, where every token at every layer selects FullAttn, `SwiAttn` has the same attention and KV-cache access complexity as FullAttn together with the small router cost, and therefore provides no speedup.

Acknowledgements

This paper is partially supported by the National Key Research and Development Program of China with Grant No. 2023YFC3341203 as well as the National Natural Science Foundation of China with Grant Number 62276002.

References

Bilge Acun, Prasoon Sinha, Newsha Ardalani, Sangmin Bae, Alicia Golden, Chien-Yu Lin, Meghana Madhyastha, Fei Sun, Neeraja J Yadwadkar, and Carole-Jean Wu. 2025. Composer: A search framework for hybrid neural architecture design. *arXiv preprint arXiv:2510.00379*.

Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901.

Alexander A Alemi, Ian Fischer, Joshua V Dillon, and Kevin Murphy. 2016. Deep variational information bottleneck. *arXiv preprint arXiv:1612.00410*.

Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, and 1 others. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.

Simran Arora, Aman Timalsina, Aaryan Singhal, Benjamin Spector, Sabri Eyuboglu, Xinyi Zhao, Ashish Rao, Atri Rudra, and Christopher Ré. 2024. Just read twice: closing the recall gap for recurrent language models. *arXiv preprint arXiv:2407.05483*.

Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. 2023. Language models enable simple systems for generating structured views of heterogeneous data lakes. *arXiv preprint arXiv:2304.09433*.

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, and 1 others. 2024. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 3119–3137.

Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*.

Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, and 1 others. 2020. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and 1 others. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*.

Yang-Rwei Chang, Hsuan-Fu Chen, and Horng-Yuan Shih. 2025. Mosa: matrix optimized self-attention hardware accelerator for mobile device. In *2025 International Conference on Electronics, Information, and Communication (ICEIC)*, pages 1–4. IEEE.

Yaofo Chen, Zeng You, Shuhai Zhang, Haokun Li, Yirui Li, Yaowei Wang, and Mingkui Tan. 2025. Core context aware transformers for long context language modeling. In *Forty-second International Conference on Machine Learning*.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*.

Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. Boolq: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 conference of the north American chapter of the association for computational linguistics: Human language technologies, volume 1 (long and short papers)*, pages 2924–2936.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*.

Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le, and Ruslan Salakhutdinov. 2019. Transformer-xl: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 2978–2988.

Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359.

Tri Dao and Albert Gu. 2024. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*.

Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A Smith, and Matt Gardner. 2021. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4599–4610.

Jusen Du, Jiaxi Hu, Tao Zhang, Weigao Sun, and Yu Cheng. 2025. Native hybrid attention for efficient sequence modeling. *arXiv preprint arXiv:2510.07019*.

Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. 2019. Drop: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2368–2378.

Alexander Richard Fabbri, Irene Li, Tianwei She, Suyi Li, and Dragomir Radev. 2019. Multi-news: A large-scale multi-document summarization dataset and abstractive hierarchical model. In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 1074–1084.

William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39.

Tianyu Fu, Yi Ge, Yichen You, Enshu Liu, Zhihang Yuan, Guohao Dai, Shengen Yan, Huazhong Yang, and Yu Wang. 2025. R2r: Efficiently navigating

divergent reasoning paths with small-large model token routing. *arXiv preprint arXiv:2505.21600*.

Tianyu Fu, Haofeng Huang, Xuefei Ning, Genghan Zhang, Boju Chen, Tianqi Wu, Hongyi Wang, Zixiao Huang, Shiyao Li, Shengen Yan, and 1 others. 2024. Moa: Mixture of sparse attention for automatic large language model compression. *arXiv preprint arXiv:2406.14909*.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, and 5 others. 2024. The language model evaluation harness.

Yizhao Gao, Jianyu Wei, Qihao Zhang, Yu Cheng, Shimao Chen, Zhengju Tang, Zihan Jiang, Yifan Song, Hailin Zhang, Liang Zhao, and 1 others. 2026. Hysparse: A hybrid sparse attention architecture with oracle token selection and kv cache sharing. *arXiv preprint arXiv:2602.03560*.

Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2023. Model tells you what to discard: Adaptive kv cache compression for llms. *arXiv preprint arXiv:2310.01801*.

Team Gemma. 2024. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*.

Bogdan Gliwa, Iwona Mochol, Maciej Biesek, and Aleksander Wawer. 2019. Samsum corpus: A human-annotated dialogue dataset for abstractive summarization. In *Proceedings of the 2nd Workshop on New Frontiers in Summarization*, pages 70–79.

Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. 2024. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*.

Albert Gu and Tri Dao. 2024. Mamba: Linear-time sequence modeling with selective state spaces. In *First conference on language modeling*.

Yuxian Gu, Qinghao Hu, Shang Yang, Haocheng Xi, Junyu Chen, Song Han, and Han Cai. 2025. Jet-nemotron: Efficient language model with post neural architecture search. *ArXiv*, abs/2508.15884.

Han Guo, Songlin Yang, Tarushii Goel, Eric P Xing, Tri Dao, and Yoon Kim. 2025a. Log-linear attention. *arXiv preprint arXiv:2506.04761*.

Yiduo Guo, Jie Fu, Huishuai Zhang, and Dongyan Zhao. 2025b. Efficient domain continual pretraining by mitigating the stability gap. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32850–32870.

Dongchen Han, Xuran Pan, Yizeng Han, Shiji Song, and Gao Huang. 2023. Flatten transformer: Vision transformer using focused linear attention. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 5961–5971.

Dongchen Han, Ziyi Wang, Zhuofan Xia, Yizeng Han, Yifan Pu, Chunjiang Ge, Jun Song, Shiji Song, Bo Zheng, and Gao Huang. 2024. Demystify mamba in vision: A linear attention perspective. *Advances in neural information processing systems*, 37:127181–127203.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. Ruler: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*.

Luyang Huang, Shuyang Cao, Nikolaus Parulian, Heng Ji, and Lu Wang. 2021. Efficient attentions for long document summarization. In *Proceedings of the 2021 conference of the north American chapter of the association for computational linguistics: Human language technologies*, pages 1419–1436.

Minyoung Huh, Brian Cheung, Pulkit Agrawal, and Phillip Isola. 2023. Straightening out the straight-through estimator: Overcoming optimization challenges in vector quantized networks. In *International Conference on Machine Learning*, pages 14096–14113. PMLR.

Albert Qiaochu Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7b. *ArXiv*, abs/2310.06825.

Huiqiang Jiang, YUCHENG LI, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR.

Team Kimi, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, Zhiyuan Li, and 1 others. 2025. Kimi

linear: An expressive, efficient attention architecture. *arXiv preprint arXiv:2510.26692*.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. 2020. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, and 1 others. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626.

Xunhao Lai, Jianqiao Lu, Yao Luo, Yiyuan Ma, and Xun Zhou. 2025. Flexprefill: A context-aware sparse attention mechanism for efficient long-sequence inference. *arXiv preprint arXiv:2502.20766*.

Xunhao Lai, Weiqi Xu, Yufeng Yang, Qiaorui Chen, Yang Xu, Lunbin Zeng, Xiaolong Li, Haohai Sun, Haichao Zhu, Vito Zhang, and 1 others. 2026. Minimax sparse attention. *arXiv preprint arXiv:2606.13392*.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR.

Xin Li and Dan Roth. 2002. Learning question classifiers. In *COLING 2002: The 19th International Conference on Computational Linguistics*.

Yuping Lin, Zitao Li, Yue Xing, Pengfei He, Yingqian Cui, Yaliang Li, Bolin Ding, Jingren Zhou, and Jiliang Tang. 2026. Retrieval heads are dynamic. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15710–15729.

Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, and 1 others. 2024a. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*.

Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, and 1 others. 2025. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*.

Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. 2024b. Minicache: Kv cache compression in depth dimension for large language models. *Advances in Neural Information Processing Systems*, 37:139997–140031.

Liu Liu, Zheng Qu, Zhaodong Chen, Fengbin Tu, Yufei Ding, and Yuan Xie. 2022a. Dynamic sparse attention for scalable transformer acceleration. *IEEE Transactions on Computers*, 71(12):3165–3178.

Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, and 1 others. 2024c. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 38–56.

Yuxin Liu and Ruiyi Wang. 2025. Differentiable neural architecture search based on hybrid attention for classification. In *2025 4th International Symposium on Computer Applications and Information Technology (ISCAIT)*, pages 204–207. IEEE.

Zechun Liu, Kwang-Ting Cheng, Dong Huang, Eric P Xing, and Zhiqiang Shen. 2022b. Nonuniform-to-uniform quantization: Towards accurate quantization via generalized straight-through estimation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4942–4952.

Colin Lockard, Prashant Shiralkar, and Xin Luna Dong. 2019. Openceres: When open information extraction meets the semi-structured web. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 3047–3056.

Xuezhe Ma, Xiang Kong, Sinong Wang, Chunting Zhou, Jonathan May, Hao Ma, and Luke Zettlemoyer. 2021. Luna: Linear unified nested attention. *Advances in Neural Information Processing Systems*, 34:2441–2453.

Matías Mendieta, Boran Han, Xingjian Shi, Yi Zhu, and Chen Chen. 2023. Towards geospatial foundation models via continual pretraining. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 16806–16816.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*.

Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 2381–2391.

Vinod Nair and Geoffrey E Hinton. 2010. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th international conference on machine learning (ICML-10)*, pages 807–814.

Piotr Nawrot, Robert Li, Renjie Huang, Sebastian Ruder, Kelly Marchisio, and Edoardo M Ponti. 2025. The sparse frontier: Sparse attention trade-offs in transformer llms. *arXiv preprint arXiv:2504.17768*.

Philipp Nazari and T Konstantin Rusch. 2026. The key to state reduction in linear attention: A rank-based perspective. *arXiv preprint arXiv:2602.04852*.

OpenAI. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.

OpenAI. 2025. gpt-oss-120b & gpt-oss-20b model card. *CoRR*, abs/2508.10925.

Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc-Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. 2016. The lambada dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 1525–1534.

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Leon Derczynski, and 1 others. 2023a. Rwkv: Reinventing rnns for the transformer era. In *Findings of the association for computational linguistics: EMNLP 2023*, pages 14048–14077.

Bo Peng, Ruichong Zhang, Daniel Goldstein, Eric Alcaide, Xingjian Du, Haowen Hou, Jiaju Lin, Jiaxing Liu, Janna Lu, William Merrill, and 1 others. 2025. Rwkv-7" goose" with expressive dynamic state evolution. *arXiv preprint arXiv:2503.14456*.

Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. 2023b. Yarn: Efficient context window extension of large language models. *arXiv preprint arXiv:2309.00071*.

Pranav Rajpurkar, Robin Jia, and Percy Liang. 2018. Know what you don't know: Unanswerable questions for squad. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 784–789.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106.

Rya Sanovar, Srikant Bharadwaj, Renee St Amant, Victor Rühle, and Saravan Rajmohan. 2025. Leanattention: Hardware-aware scalable attention mechanism for the decode-phase of transformers. *Proceedings of Machine Learning and Systems*, 7.

Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi. 2019. Social iqa: Commonsense reasoning about social interactions. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 4463–4473.

Noam Shazeer. 2019. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*.

Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, and 1 others. 2025. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*.

Lin Song, Yukang Chen, Shuai Yang, Xiaohan Ding, Yixiao Ge, Ying-Cong Chen, and Ying Shan. 2024. Low-rank approximation for sparse attention in multimodal llms. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 13763–13773.

Ao Sun, Hongtao Zhang, Heng Zhou, Yixuan Ma, Yiran Qin, Tongrui Su, Yan Liu, Zhanyu Ma, Jun Xu, Jiuchong Gao, and 1 others. 2026. State rank dynamics in linear attention llms. *arXiv preprint arXiv:2602.02195*.

Yu Sun, Shuohuan Wang, Yukun Li, Shikun Feng, Hao Tian, Hua Wu, and Haifeng Wang. 2020. Ernie 2.0: A continual pre-training framework for language understanding. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 8968–8975.

Yutao Sun, Zhenyu Li, Yike Zhang, Tengyu Pan, Bowen Dong, Yuyi Guo, and Jianyong Wang. 2025. Efficient attention mechanisms for large language models: A survey. *arXiv preprint arXiv:2507.19595*.

Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. 2022. Efficient transformers: A survey. *ACM Computing Surveys*, 55(6):1–28.

Naftali Tishby, Fernando C Pereira, and William Bialek. 2000. The information bottleneck method. *arXiv preprint physics/0004057*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.

Aladin Virmaux and Kevin Scaman. 2018. Lipschitz regularity of deep neural networks: analysis and efficient estimation. *Advances in neural information processing systems*, 31.

Dustin Wang, Rui-Jie Zhu, Steven Abreu, Yong Shan, Taylor Kergan, Yuqi Pan, Yuhong Chou, Zheng Li, Ge Zhang, Wenhao Huang, and 1 others. 2025. A systematic analysis of hybrid linear attention. *arXiv preprint arXiv:2507.06457*.

Wenhao Wu, Yizhong Wang, Guangxuan Xiao, Hao Peng, and Yao Fu. 2025. Retrieval head mechanistically explains long-context factuality. In *The Thirteenth International Conference on Learning Representations*.

Xiaojie Xia, Huigang Zhang, Chaoliang Zhong, Jun Sun, and Yusuke Oishi. 2026. Distill-then-replace: Efficient task-specific hybrid attention model construction. *arXiv preprint arXiv:2601.11667*.

Bangjun Xiao, Bingquan Xia, Bo Yang, Bofei Gao, Bowen Shen, Chen Zhang, Chenhong He, Chiheng Lou, Fuli Luo, Gang Wang, and 1 others. 2026. Mimo-v2-flash technical report. *arXiv preprint arXiv:2601.02780*.

Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. 2025. Duoattention: Efficient long-context llm inference with retrieval and streaming heads. In *International Conference on Learning Representations*, volume 2025, pages 37228–37253.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*.

Ruyi Xu, Guangxuan Xiao, Haofeng Huang, Junxian Guo, and Song Han. 2025. Xattention: Block sparse attention with antidiagonal scoring. *arXiv preprint arXiv:2503.16428*.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2024. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*.

Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and 1 others. 2025. Flashinfer: Efficient and customizable attention engine for llm inference serving. *Proceedings of Machine Learning and Systems*, 7.

Çağatay Yıldız, Nishaanth Kanna Ravichandran, Nitin Sharma, Matthias Bethge, and Beyza Ermis. 2024. Investigating continual pretraining in large language models: Insights and implications. *arXiv preprint arXiv:2402.17400*.

Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Yuxing Wei, Lean Wang, Zhiping Xiao, and 1 others. 2025. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23078–23097.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and 1 others. 2020. Big bird: Transformers for longer sequences. *Advances in neural information processing systems*, 33:17283–17297.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 4791–4800.

Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, and 1 others. 2026. Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*.

Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization. *Advances in neural information processing systems*, 32.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, and 1 others. 2023. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36:34661–34710.

Algorithm 2 The inference procedure of `SwiAttn`

Requires: The `SwiAttn` model $\mathcal{M}$, the prompt P.
Ensures: The response sequence R.

Tokenize P and convert it to the initial hidden representations $\boldsymbol{H}^{(1)}$;
for Layer $l = 1, 2, ..., L$ **do**
  Compute $\boldsymbol{Q}^{(l)}, \boldsymbol{K}^{(l)}, \boldsymbol{V}^{(l)}$ and initialize the shared KV-cache $\mathcal{C}^{(l)} = \{\boldsymbol{k}_i^{(l)}, \boldsymbol{v}_i^{(l)}\}_{i=1}^{t-1}$;
  Compute the gate (Eq.1-2), attention outputs (Eq.4-6), and apply the FFN layer.
end for
Generate the first token and append to R;
while not reaching EOS token **do**
  for Layer $l = 1, 2, ..., L$ **do**
    Compute $\boldsymbol{q}_t^{(l)}, \boldsymbol{k}_t^{(l)}, \boldsymbol{v}_t^{(l)}$ and update $\mathcal{C}^{(l)}$;
    Compute the gate $\sigma_t^{(l)}$ using Eq.1-2;
    if $\sigma_t^{(l)} = 1$ **then**
      Compute full attention output $\boldsymbol{o}^{(l,t)} = \boldsymbol{o}_{\text{FULL}}^{(l,t)} = \mathcal{A}_{\text{FULL}}^{(l)}(\boldsymbol{q}_t^{(l)}, \mathcal{C}^{(l)})$;
    else
      Compute the SWA output $\boldsymbol{o}^{(l,t)} = \boldsymbol{o}_{\text{SWA}}^{(l,t)} = \mathcal{A}_{\text{SWA}}^{(l)}(\boldsymbol{q}_t^{(l)}, \mathcal{C}^{(l)})$;
    end if
    Apply the FFN layer to $\boldsymbol{o}^{(l,t)}$;
  end for
  Generate the next token and append to R;
end while
return The detokenized sequence R;

A Inference Procedure of `SwiAttn`

We provide the inference procedure of the proposed `SwiAttn` in Algorithm 2.

B Details about the Kernel

The prefill implementation compacts queries routed to the FullAttn and SWA branches into separate, position-sorted lists and tiles them across query positions. This decouples arithmetic intensity from the GQA group size, avoiding a constraint common to many sparse-attention kernels. Figure 7 provides a compact TileLang implementation.

C Motivation for Adaptive Regularization

The adaptive weight in Eq.8 is motivated by two complementary intuitions. First, if the mapping from the l-th layer output to the final language-modeling loss is locally Lipschitz continuous (Virmaux and Scaman, 2018), a larger discrepancy between the FullAttn and SWA outputs may correspond to a larger change in the downstream loss. This motivates using branch disagreement as a signal for weakening the efficiency-oriented penalty:

$$|\mathcal{L}_{\text{LM}}(\boldsymbol{o}_{\text{SWA}}^{(t,l)}) - \mathcal{L}_{\text{LM}}(\boldsymbol{o}_{\text{FULL}}^{(t,l)})| \leq K\|\boldsymbol{o}_{\text{FULL}}^{(t,l)} - \boldsymbol{o}_{\text{SWA}}^{(t,l)}\|_2.$$

Second, the information-bottleneck perspective (Tishby et al., 2000; Alemi et al., 2016) views token-level NLL as an indicator of prediction difficulty and information demand. A higher NLL therefore motivates preserving greater flexibility to access global context. Combining NLL with branch disagreement allows the regularization strength to adapt to both token difficulty and the difference between the two attention branches, encouraging SWA for locally predictable tokens while retaining FullAttn as an option when local information may be insufficient.

D Details about the Datasets

In this section, we provide comprehensive descriptions of the datasets used in our experiments, organized by the three evaluation categories: commonsense reasoning, in-context retrieval, and long-context understanding.

D.1 Commonsense Reasoning

This category evaluates models' ability to perform reasoning based on everyday world knowledge.

- **Lambada (LMB.)** (Paperno et al., 2016): A word prediction task requiring broad discourse context understanding. Models must predict the final word of a passage, which cannot be inferred from local context alone. The dataset contains passages extracted from BookCorpus. We report perplexity on the standard test set.
- **Wikitext (Wiki.)** (Merity et al., 2016): A language modeling dataset derived from Wikipedia articles. We evaluate perplexity on the test split to assess generative capability over coherent, long-form text.
- **BoolQ** (Clark et al., 2019): A binary question answering dataset where questions are naturally occurring yes/no queries from Google search. Evaluation is based on exact match accuracy.
- **PIQA** (Bisk et al., 2020): Physical Interaction QA, focusing on everyday physical reasoning. Models choose between two plausible solutions to a goal. Accuracy is reported on the test set.

```python
import tilelang
from tilelang import language as T

PASS_CONFIGS = {
    tilelang.PassConfigKey.TL_DISABLE_TMA_LOWER: True,
    tilelang.PassConfigKey.TL_DISABLE_WARP_SPECIALIZED: True,
    tilelang.PassConfigKey.TL_ENABLE_FAST_MATH: True,
}

@tilelang.jit(pass_configs=PASS_CONFIGS)
def switch_gqa_prefill_branch(
    Q, K, V, QueryIndices, QueryCount, Output, Lse,
    heads, dim, num_queries, kv_group, window_size,
    sm_scale=None, is_local=False, block_M=64, block_K=64,
    num_stages=2, threads=128,
):
    assert dim == 128 and heads % kv_group == 0
    assert block_M in (64, 128) and block_K in (64, 128)
    assert num_queries % block_M == 0 and window_size > 0
    sm_scale = ((1.0 / dim) ** 0.5 if sm_scale is None else sm_scale) * 1.4426950408889634
    B, S, S_kv = T.dynamic("batch"), T.dynamic("seq_len"), T.dynamic("seq_len_kv")
    BF16, F32, I32 = T.bfloat16, T.float32, T.int32
    BM, BK, D = block_M, block_K, dim
    NM, heads_per_kv = num_queries // BM, heads // kv_group

    Q: T.Tensor([B, S, heads, D], BF16); K: T.Tensor([B, S_kv, kv_group, D], BF16)
    V: T.Tensor([B, S_kv, kv_group, D], BF16)
    QueryIndices: T.Tensor([B, num_queries], I32); QueryCount: T.Tensor([B], I32)
    Output: T.Tensor([B, S, heads, D], BF16); Lse: T.Tensor([B, S, heads], F32)

    with T.Kernel(NM, B, heads, threads=threads) as (bx, by, bz):
        Qs = T.alloc_shared([BM, D], BF16)
        Ks, Vs = T.alloc_shared([BK, D], BF16), T.alloc_shared([BK, D], BF16)
        Ps, Os = T.alloc_shared([BM, BK], BF16), T.alloc_shared([BM, D], BF16)
        qpos, valid = T.alloc_fragment([BM], I32), T.alloc_fragment([BM], "bool")
        acc_s = T.alloc_fragment([BM, BK], F32); acc_o = T.alloc_fragment([BM, D], F32)
        row_max = T.alloc_fragment([BM], F32); row_max_prev = T.alloc_fragment([BM], F32)
        row_sum = T.alloc_fragment([BM], F32); row_sum_block = T.alloc_fragment([BM], F32)
        alpha = T.alloc_fragment([BM], F32)

        b, h, kv = by, bz, bz // heads_per_kv
        q0, count = bx * BM, QueryCount[by]
        edge = T.max(count - 1, 0)
        first_slot, last_slot = T.min(q0, edge), T.min(q0 + BM - 1, edge)
        first_q = T.max(QueryIndices[b, first_slot], 0)
        last_q = T.max(QueryIndices[b, last_slot], 0)
        first_key = T.max(first_q - window_size + 1, 0) if is_local else 0
        first_block = first_key // BK
        last_block = T.ceildiv(T.min(last_q + 1, S_kv), BK)
        num_blocks = T.max(last_block - first_block, 0)

        for mi in T.Parallel(BM):
            slot = q0 + mi; valid[mi] = slot < count
            qpos[mi] = T.if_then_else(valid[mi], QueryIndices[b, slot], 0)
        for mi in T.serial(BM):
            q = T.max(0, T.min(qpos[mi], S - 1))
            T.copy(Q[b, q, h, :], Qs[mi, :], disable_tma=True)

        T.fill(acc_o, 0); T.fill(row_sum, 0); T.fill(row_max, -(2**30))
        for jb in T.Pipelined(num_blocks, num_stages=num_stages):
            key0 = (first_block + jb) * BK
            T.copy(K[b, key0:key0 + BK, kv, :], Ks, disable_tma=True)
            for mi, ni in T.Parallel(BM, BK):
                key_pos = key0 + ni
                lower = T.max(qpos[mi] - window_size + 1, 0) if is_local else 0
                legal = valid[mi] and key_pos < S_kv and key_pos >= lower and key_pos <= qpos[mi]
                acc_s[mi, ni] = T.if_then_else(legal, 0, -T.infinity(F32))

            T.wgmma_gemm(Qs, Ks, acc_s, transpose_B=True, policy=T.GemmWarpPolicy.FullRow)
            T.wait_wgmma(0); T.copy(row_max, row_max_prev)
            T.reduce_max(acc_s, row_max, dim=1, clear=False)
            for mi in T.Parallel(BM):
                row_max[mi] = T.max(row_max[mi], row_max_prev[mi])
                alpha[mi] = T.exp2((row_max_prev[mi] - row_max[mi]) * sm_scale)
            for mi, ni in T.Parallel(BM, BK):
                acc_s[mi, ni] = T.exp2((acc_s[mi, ni] - row_max[mi]) * sm_scale)
            T.reduce_sum(acc_s, row_sum_block, dim=1)
            for mi in T.Parallel(BM):
                row_sum[mi] = row_sum[mi] * alpha[mi] + row_sum_block[mi]
            for mi, di in T.Parallel(BM, D): acc_o[mi, di] *= alpha[mi]
            T.copy(acc_s, Ps); T.copy(V[b, key0:key0 + BK, kv, :], Vs, disable_tma=True)
            T.wgmma_gemm(Ps, Vs, acc_o, policy=T.GemmWarpPolicy.FullRow); T.wait_wgmma(0)

        for mi, di in T.Parallel(BM, D): acc_o[mi, di] /= row_sum[mi]
        for mi in T.Parallel(BM):
            row_sum[mi] = T.log2(row_sum[mi]) + row_max[mi] * sm_scale
        T.copy(acc_o, Os)
        for mi, di in T.Parallel(BM, D):
            if valid[mi]: Output[b, qpos[mi], h, di] = Os[mi, di]
        for mi in T.Parallel(BM):
            if valid[mi]: Lse[b, qpos[mi], h] = row_sum[mi]
```

Figure 7: Compact TileLang implementation of the branch-specific **SwiAttn** prefill kernel.

- **SIQA** (Sap et al., 2019): Social Interaction QA, assessing understanding of social commonsense. Models select the most appropriate response to a given situation among candidates.
- **HellaSwag (Hella.)** (Zellers et al., 2019): A challenging dataset for commonsense natural language inference (NLI), requiring models to predict the most plausible continuation of an event description among four candidates. Normalized accuracy is evaluated on the test split.
- **WinoGrande (Wino.)** (Sakaguchi et al., 2021): A 44K-example Winograd schema dataset testing pronoun disambiguation with adversarial filtering. We report binary accuracy.
- **ARC-easy & ARC-challenge (ARC-e/c)** (Clark et al., 2018): The AI2 Reasoning Challenge, containing grade-school science questions. ARC-e consists of relatively straightforward questions, while ARC-c includes more challenging ones requiring multi-step reasoning. Accuracy is reported for ARC-e, while normalized accuracy is reported for ARC-c.
- **OpenBookQA (OBQA)** (Mihaylov et al., 2018): A dataset requiring external commonsense knowledge beyond the provided "open book" fact. Models answer multiple-choice science questions; evaluation is based on normalized accuracy.

D.2 In-Context Retrieval

Following Arora et al. (2024), this category assesses models' ability to retrieve and utilize relevant information from a maximum of $4,096$ token context. We use Cloze Completion Formatting prompts aligned with next-word prediction.

- **SWDE** (Lockard et al., 2019): A structured extraction dataset where models answer questions by retrieving attribute values from semi-structured web pages (HTML).
- **FDA** (Arora et al., 2023): Food and Drug Administration document QA, requiring extraction of specific regulatory information from PDF documents.
- **TriviaQA (TQA)** (Joshi et al., 2017): A large-scale reading comprehension dataset with question-answer-evidence triples.
- **Natural Questions (NQ)** (Kwiatkowski et al., 2019): Real user queries to Google Search with answers annotated on Wikipedia pages.
- **DROP** (Dua et al., 2019): A dataset requiring discrete reasoning over numerical and temporal information. Models generate answers via span extraction, counting, or arithmetic.
- **SQuAD (SQD)** (Rajpurkar et al., 2018): Stanford Question Answering Dataset, where answers are text spans from Wikipedia passages.

D.3 Long-Context Understanding

We adopt benchmark datasets from LongBench-E (Bai et al., 2024) to evaluate performance on tasks requiring processing of extended contexts. We focus on single-document QA, summarization, and few-shot learning, which cover natural-language document understanding and in-context learning. Specifically, we include MultiFieldQA-en, Qasper, GovReport, MultiNews, TriviaQA, SAMSum, and TREC. We exclude the multi-document QA datasets HotpotQA and 2WikiMultihopQA, the synthetic datasets PassageCount and PassageRetrieval-en, and the code-completion datasets LCC and RepoBench-P. The included datasets are described below.

- **MultiFieldQA (MFQA)** (Bai et al., 2024): MFQA is a multi-domain QA dataset with long documents spanning legal, scientific, judicial, and government fields. Answers require synthesizing information across distant passages.
- **TRec** (Li and Roth, 2002): TRec is a question classification task requiring models to categorize questions into 50 predefined topic classes.
- **MultiNews (MNs)** (Fabbri et al., 2019): MultiNews is a multi-document summarization dataset containing news articles about the same event.
- **GovReport (GvR)** (Huang et al., 2021): GovReport is a long-document summarization task based on U.S. government reports and congressional hearings.
- **Qasper (Qas)** (Dasigi et al., 2021): Qasper is a QA dataset over NLP research papers, featuring questions requiring understanding of methodology, results, and claims.
- **TriviaQA (TQA)** (Joshi et al., 2017): The TriviaQA dataset in LongBench-E is a long-context variant of the original version, where relevant evidence is embedded within extended passages to test retrieval under length pressure.
- **SAMSum (SSM)** (Gliwa et al., 2019): The SAMSum dataset consists of messenger-like conversa-

tions as the input with summaries annotated by humans as the target.

D.4 Evaluation Metrics

We adopt different evaluation metrics for these datasets. Specifically, for Lambada and Wikitext, we use perplexity. For BoolQ, PIQA, SIQA, WinoGrande, and ARC-e, we use accuracy. For HellaSwag, ARC-c, and OpenBookQA, we use normalized accuracy. For in-context retrieval tasks, we use the "contains" metric and follow the prompt template of Arora et al. 2024. For LongBench-E, we use the default evaluation metric provided in Bai et al. 2024 with implementation from Gao et al. 2024.

E Details about the Baselines

In this part, we provide more details about the implementation of the baseline models.

- **SWA-ZS.** The zero-shot sliding window attention baseline is implemented using the corresponding full attention model. It directly restricts the attention span of full attention transformers to 1024 in alignment with `SwiAttn`.
- **SWA-CPT.** The continually pretrained sliding window attention model is obtained by performing CPT on the SWA-ZS model. We use the same amount of tokens as the proposed `SwiAttn` to train this baseline model.
- **SH (3:1).** This StaticHybrid baseline alternates one full attention layer after every three SWA layers. In the 22-layer transformer model, the $4, 8, 12, 16, 20$-th layers use full attention, while the remaining layers use SWA.
- **SH (6:1).** This StaticHybrid baseline uses three full attention layers and 19 SWA layers, giving an approximate SWA-to-FullAttn layer ratio of 6:1. Specifically, the $7, 14, 21$-st layers use full attention.
- **DuoAttention.** We evaluate DuoAttention (Xiao et al., 2025) using the same 1.5B backbone and LongBench-E evaluation setting.
- **FullAttn.** The full attention baseline is a 1.5B language model with 22 transformer layers. The hidden size is set to 2048. The standard version is trained on 240B tokens of 4K context lengths, while the 32K version is trained on 192B tokens of 4K lengths and 24B tokens of 32K lengths.

F Additional Implementation Details

In the experiments, we adopt a 1.5B transformer model with 22 layers and the hidden dimension of 2048. For the attention module, group query attention (Ainslie et al., 2023) is used with 16 attention heads and 8 key-value heads. The dimension of each query, key, and value vectors is set to 128. In dual-branch routing, the router function $\mathcal{R}(\cdot)$ is applied after the root-mean-square (RMS) layer normalization (Zhang and Sennrich, 2019).

Within each context-length setting, FullAttn pretraining and `SwiAttn` CPT use the same data distribution. The 4K corpus mainly contains English and Chinese text collected from web pages, books, encyclopedias, and other general sources, together with some multilingual and domain-specific data. The 32K corpus contains more long-form and instruction data; about 20% is instruction data, including diverse questions and answers with varying reasoning lengths.

For the 4K-length model, we first pretrain the full attention transformers with 240B tokens, where we set the batch size to 1024 and train for 60000 steps. During the first 2000 steps of the pretraining, we linearly warm up the learning rate from 0 to 1×10^{-3}. During the last 12000 steps, we decay the learning rate to 0 with a cosine schedule. For the continual pretraining of `SwiAttn`, we first initialize the model as described in Section 3.4 and then continually pretrain the model for another 20B tokens with the same batch size for 5000 steps. During the CPT stage, we linearly warm up the learning rate from 0 to 1×10^{-4} for 500 steps and decay the learning rate in the remaining 4500 steps with a cosine schedule.

For the 32K-length version of the model, we first pretrain the full attention transformer model for 192B tokens on 4K-length text, and then extend the context length to 32K using YaRN (Peng et al., 2023b). Then we train the full attention model on 32K-length texts for another 24B tokens with the batch size of 128 for 6000 steps. During the first 2000 steps, the learning rate linearly warms up to 1×10^{-3}, and during the last 24B token on the 32K-length texts (*i.e.*, last 6000 steps), the learning rate is decayed to 0 under a cosine schedule. The 32K-length version of `SwiAttn` is first initialized using the aforementioned full attention model as described in Section 3.4 and then continually pretrained for another 8B tokens with the same batch size (*i.e.*, 128) for another 2000 steps. In the first

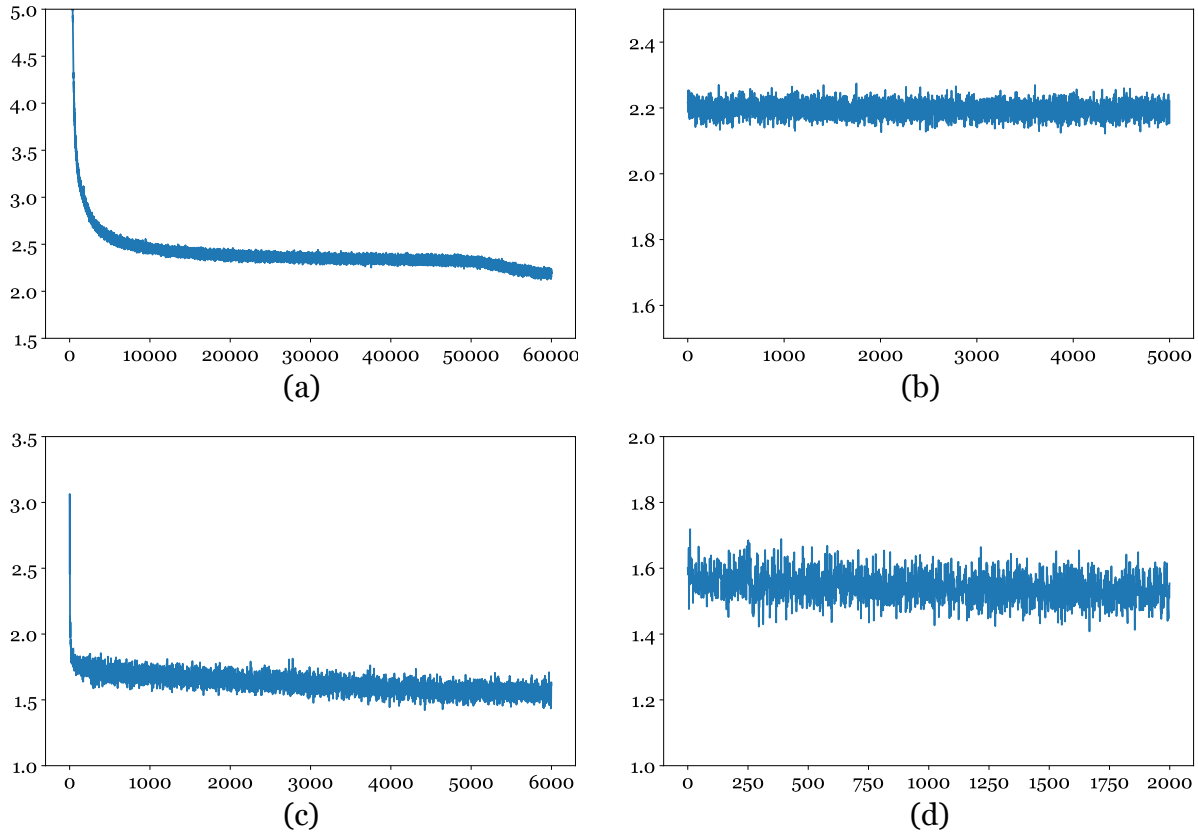


Figure 8: Language modeling loss curves of (a) pretraining of the full attention transformer on 4K-length sequences, (b) continual pretraining of `SwiAttn` on 4K-length sequences, (c) pretraining of the full attention transformer on 32K-length sequences, and (d) continual pretraining of `SwiAttn` on 32K-length sequences.

200 steps, we warm up the learning rate from 0 to 2×10^{-4}, and in the remaining 1800 steps, we decay the learning rate to 0 using a cosine schedule. The additional CPT budgets correspond to 8.3% of the preceding 4K pretraining budget and 3.7% of the preceding 32K model-training budget. CPT is used to adapt the architecture and learn the newly introduced router, and SWA-CPT and the StaticHybrid baselines receive the same additional token budget as `SwiAttn`.

We use Megatron-LM (Shoeybi et al., 2019) to train the transformer model and adopt LM-Eval-Harness (Gao et al., 2024) to evaluate the models on various datasets. By default, the results are the models' performance under a single run. For the NIAH experiments, the RULER (Hsieh et al., 2024) setup is applied. All experiments are performed on Ascend 910C NPUs.

G Loss Curves

In this part, we provide the loss curves of model optimization. Specifically, we visualize the language modeling loss of (a) pretraining of the full attention transformer on 4K-length sequences; (b) continual pretraining of `SwiAttn` on 4K-length sequences; (c) pretraining of the full attention transformer on 32K-length sequences; (d) continual pretraining of `SwiAttn` on 32K-length sequences. The results are presented in Figure 8. These curves suggest that the optimization procedure of `SwiAttn` is relatively stable with no loss spikes. Furthermore, the transition during the continual pretraining stage of `SwiAttn` demonstrates convergence dynamics that are highly competitive with the standard baseline. This consistent stability across both 4K and 32K context windows confirms that CPT serves as a highly reliable and efficient approach for seamlessly converting standard full attention models into the proposed `SwiAttn` architecture, without sacrificing optimization integrity.

H Changes of Full Attention Ratio during Continual Pretraining

In this part, we present the changes of the full attention ratios during the continual pretraining stage. Specifically, we measure the frequency of the router

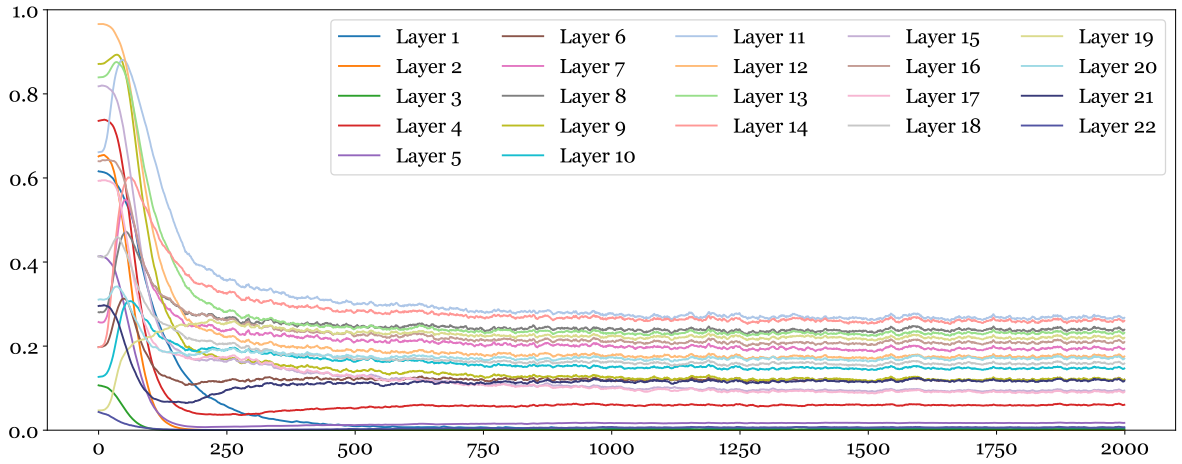


Figure 9: The changes of full attention ratio during the continual pretraining stage. The ratio is averaged within each transformer layer.

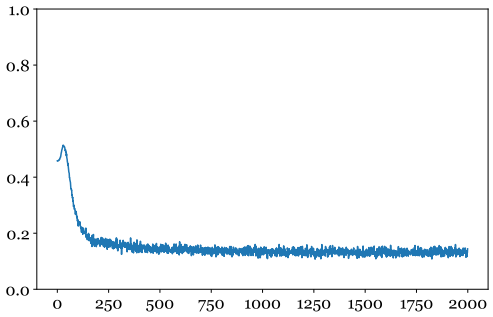


Figure 10: The changes of full attention ratio during the continual pretraining stage. The averaged ratio over all transformer layers is reported.

choosing the full attention branch during CPT on the pretraining data. The averaged ratio of full attention is presented in Figure 10, while the averaged full attention ratio of each layer is visualized in Figure 9 (where the curves are smoothed for better visualization). From the results, we can see that the proposed **SwiAttn** chooses the full attention branch with a frequency of around 0.13, which is more efficient compared to common static hybrid methods (*i.e.*, around 0.25 full attention ratio). Moreover, the per-layer statistics of **SwiAttn** suggest that different layers exhibit distinct behaviors: some layers (*e.g.*, Layer 8, 11, 13, 14) tend to use full attention more frequently, whereas others (*e.g.*, Layer 1, 2, 3, 22) tend to use SWA more frequently. Moreover, these results show that the changes of full attention ratio are relatively smooth, and that the ratio stabilizes after around 500 iterations.

I Experiments on 3B Models

We also evaluate a 3B transformer with 28 layers, hidden size 2048, 16 attention heads, 4 key-value heads, and FFN size 9216. We reuse the 1.5B training schedule and hyperparameters without a scale-specific search. Tables 5 and 6 report commonsense reasoning, in-context retrieval, and long-context understanding results for FullAttn and **SwiAttn**, providing preliminary evidence at a larger scale.

J Additional NIAH Results

We compare aggregate NIAH accuracy under the same evaluation setting in Table 7. FullAttn and **SwiAttn** both achieve perfect retrieval accuracy, while SWA and SH (3:1) obtain lower scores.

Table 7: Aggregate NIAH retrieval accuracy under the same evaluation setting.

Method	NIAH Accuracy
FullAttn	1.00
SwiAttn	1.00
SWA	0.20
SH (3:1)	0.95

K Additional Results on Efficiency

K.1 Training Overheads

Table 8: Comparison of runtime per layer during continual pretraining (CPT) with a sequence length of 32K.

Methods	Forward time (s)	Backward time (s)
FullAttn	0.0595	0.1459
SwiAttn	0.0645	0.1574
Overhead	8.4%	7.9%

Table 5: Performance of 3B models on common sense reasoning datasets.

Datasets	LMB.	Wiki.	AVG	BoolQ	PIQA	SIQA	Hella.	Wino.	ARC-e	ARC-c	OBQA	AVG
Metrics	ppl↓	ppl↓	ppl↓	acc↑	acc↑	acc↑	acc_n↑	acc↑	acc↑	acc_n↑	acc_n↑	acc↑
FullAttn	6.14	13.8	9.99	63.3	74.8	43.9	63.5	61.1	66.3	35.4	39.0	55.9
SwiAttn	6.19	13.8	9.99	65.6	74.8	43.4	63.5	61.7	65.0	34.6	38.4	55.9

Table 6: Performance of 3B models on in-context retrieval and LongBench-E datasets.

	In-Context Retrieval							LongBench-E					
Methods	SWDE	FDA	TQA	NQ	DROP	SQD	AVG	GvR	MNs	MFQA	Qas	TQA	AVG
FullAttn	65.4	76.5	65.0	38.0	34.4	53.3	55.4	27.3	21.5	39.6	27.6	70.0	37.2
SwiAttn	66.3	77.7	65.6	38.1	33.5	53.7	55.8	26.8	20.2	39.7	26.6	69.5	36.6

In this part, we analyze the computational overhead introduced during CPT. The SWA branch is computed efficiently, while the two attention branches share the QKV projections and the remaining model components, including the FFN layers. Consequently, computing both attention branches does not double the per-layer runtime.

Table 8 reports the runtime per layer during CPT at a sequence length of 32K. Compared with FullAttn, **SwiAttn** introduces 8.4% forward and 7.9% backward overhead.

K.2 Efficiency Comparison with SH (3:1)

Table 9: Efficiency comparison between **SwiAttn** and the SH (3:1) baseline.

Methods	Standard Setting		Simple Setting		LongBench-E
	Memory	GFLOPs	Memory	GFLOPs	
SH (3:1)	8.0K	1.38	8.0K	1.38	35.2
SwiAttn	**7.3K**	**1.26**	**2.6K**	**0.44**	**37.4**

We further provide an efficiency comparison between **SwiAttn** and the SH (3:1) baseline. Table 9 reports the memory access (in terms of tokens) and GFLOPs under both the standard and simple settings (as described in Section 4.4 of the main paper), alongside their average performance on the LongBench-E benchmark.

The results show that **SwiAttn** is more efficient than SH (3:1), especially in the simple setting: it reduces memory access and GFLOPs by over 67% and improves LongBench-E performance from 35.2 to 37.4.

Additionally, **SwiAttn** is complementary to existing efficiency techniques: it can replace StaticHybrid's full-attention layers or equip a sparse-attention mechanism with a router that decides whether to scan global memory.

L Statistical Significance

Table 10: Performance comparison over three runs and statistical significance (p-value) across commonsense reasoning, in-context retrieval, and LongBench-E.

Methods	Commonsense	Retrieval	LongBench
SH (3:1)	53.6±0.32	50.6±0.09	35.2±0.44
FullAttn	–	–	37.4±0.18
SwiAttn	53.7±0.30	52.9±0.11	37.4±0.35
p-value (vs. SH)	0.713	< 0.001	< 0.01
p-value (vs. FullAttn)	–	–	0.941

Table 10 reports results over three runs and the corresponding two-sided Welch's t-tests. The improvements of **SwiAttn** over SH (3:1) are statistically significant on in-context retrieval and LongBench-E. On LongBench-E, the comparison with FullAttn gives $p = 0.941$, supporting performance parity.